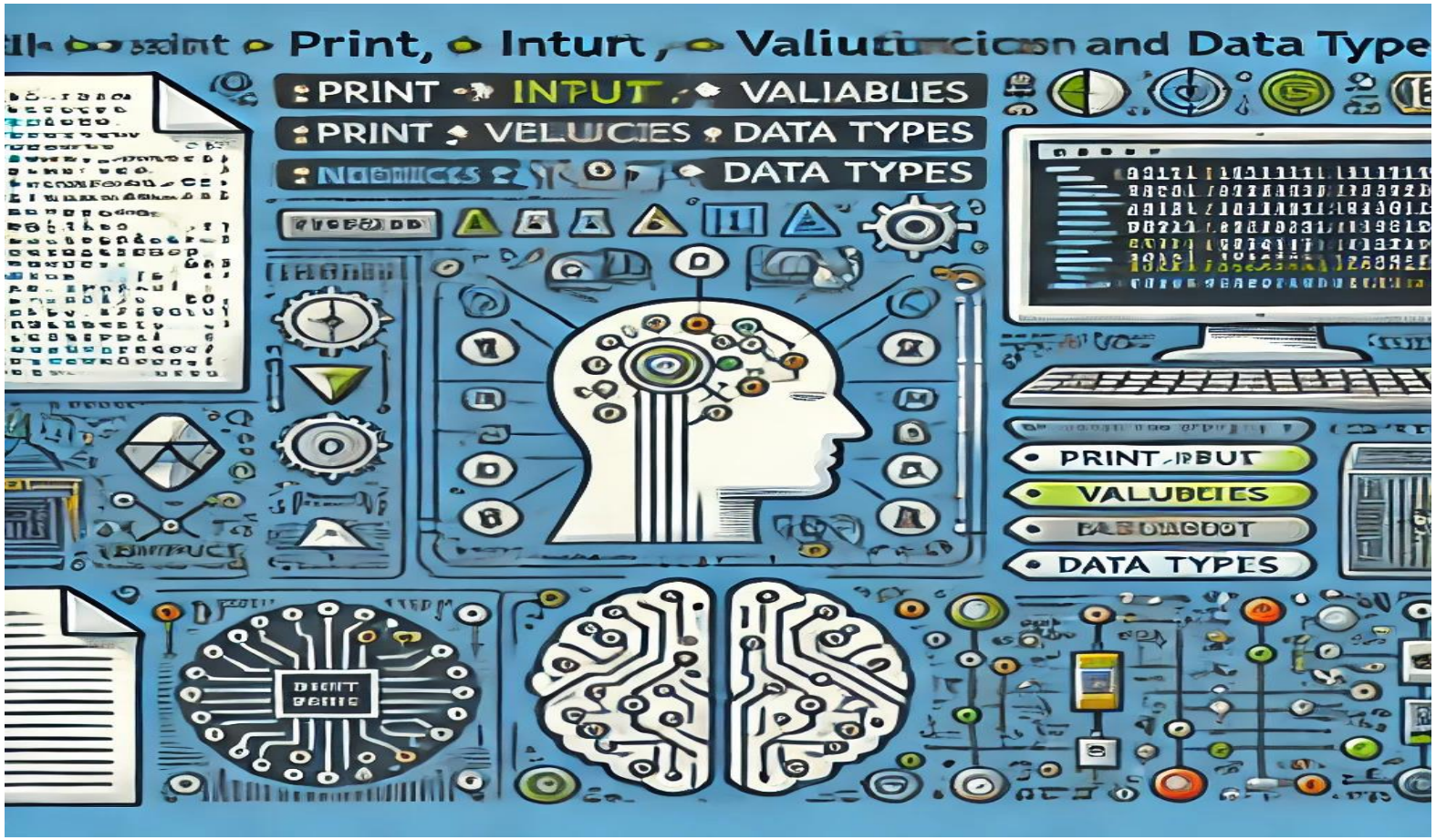
Week 1.

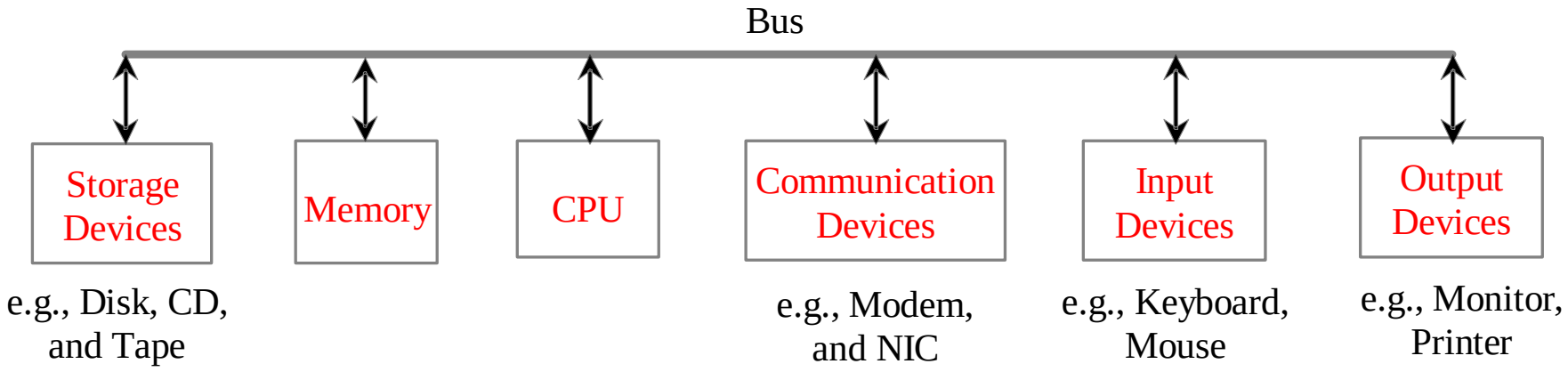

Print,
and Data Type
PRINT
INPUT
VALIABLES
PRINT
DATA TYPES
DATA TYPES
PRINT
DATA TYPES

Objectives

- **Introduction to Computer memory**
- **Elementary Programming**
  - **Print**
  - **Input**
  - **Variable**

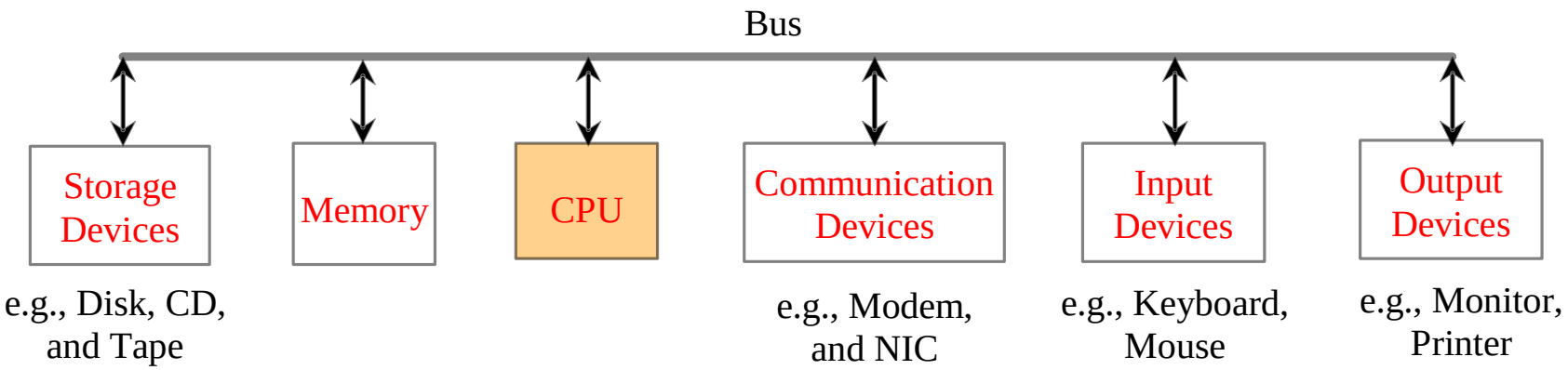
Computer

A computer consists of a CPU, memory, hard disk, input/output devices, and communication devices.



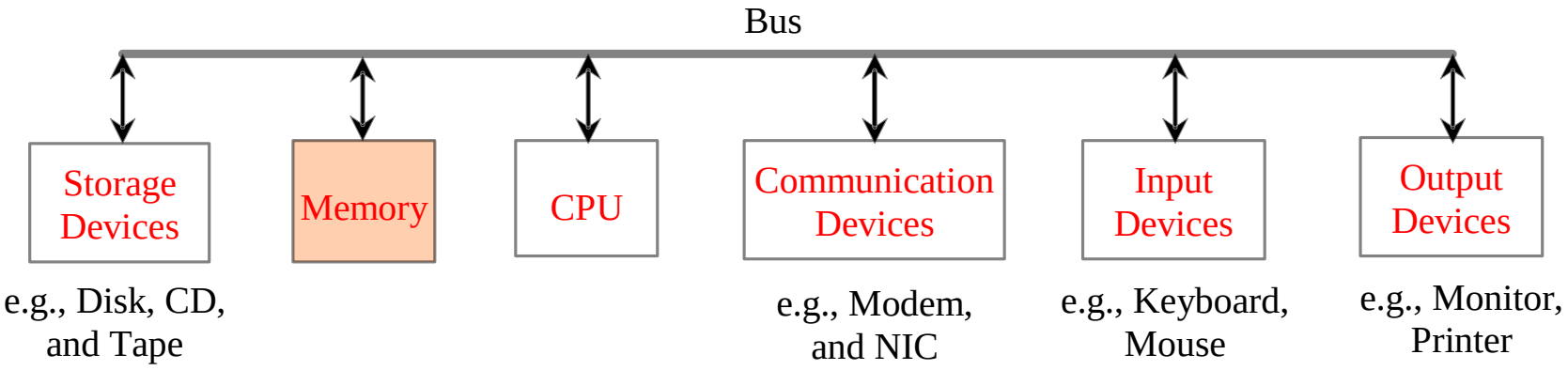
CPU

The central processing unit (CPU) is the brain of a computer. It retrieves instructions from memory and executes them



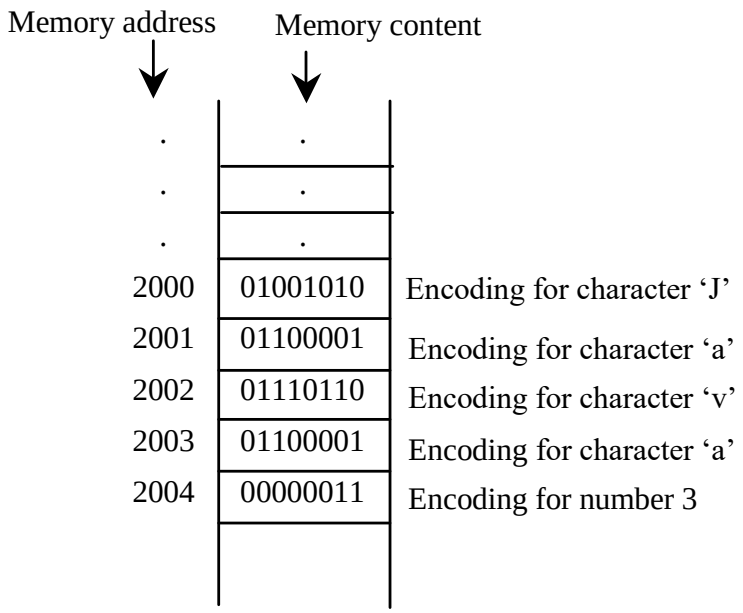
Memory

Memory is to store data and program instructions for CPU to execute. A memory unit is an ordered sequence of bytes.



How data is stored?

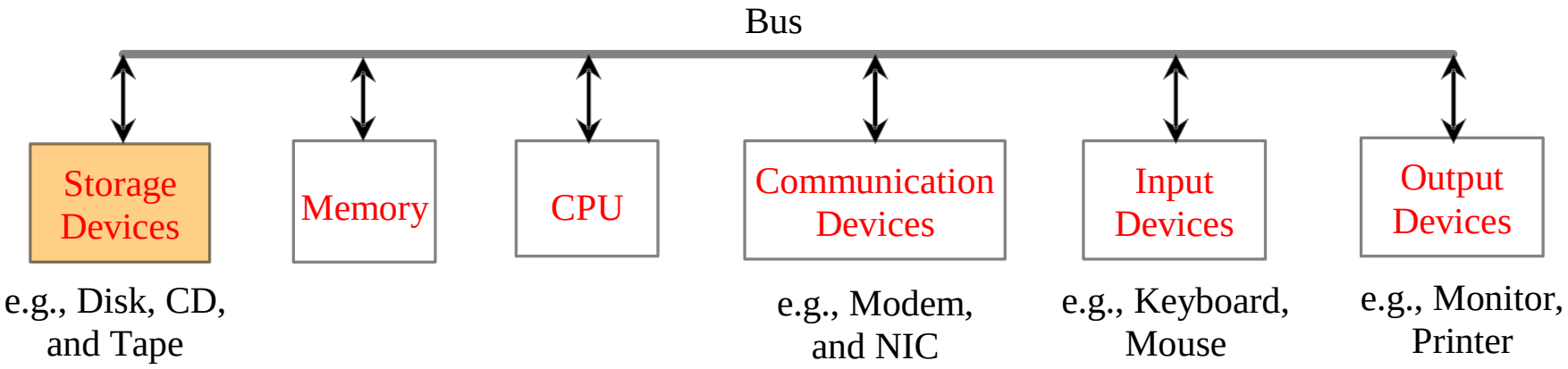
- Data of various kinds, such as numbers, characters, and strings, are encoded as a series of bits (0 and 1).
- Computers use zeros and ones because digital devices have two stable states, which are referred to as *zero* and *one* by convention.
- YOU need not to be concerned about the encoding and decoding of data, which is performed automatically by the system based on the encoding scheme
- No two data can share or split a same byte. A byte is the minimum storage unit.



Storage Devices

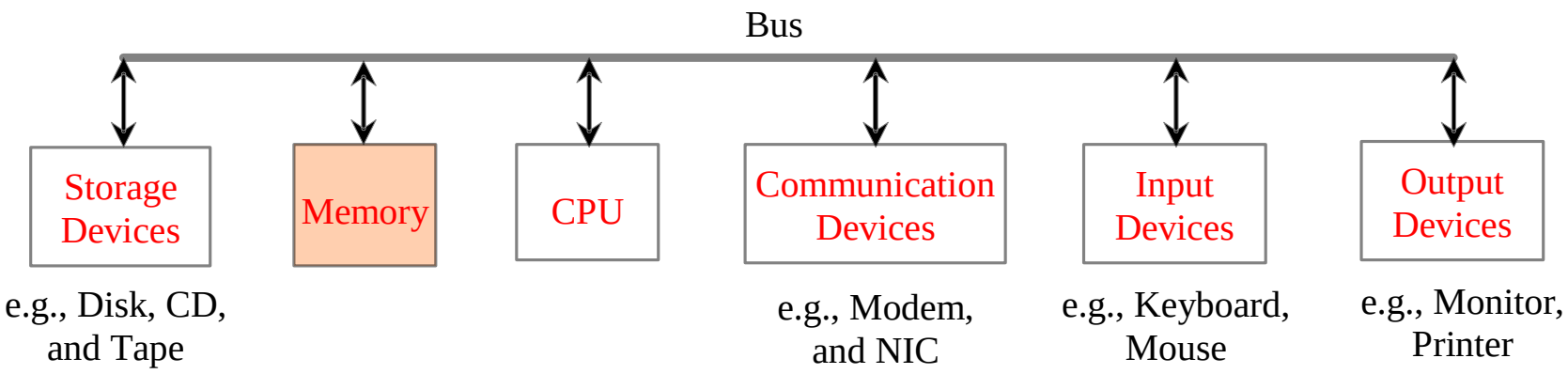
Memory is volatile, because information is lost when the power is off.

Programs and data are permanently stored on storage devices and are moved to memory when the computer actually uses them.



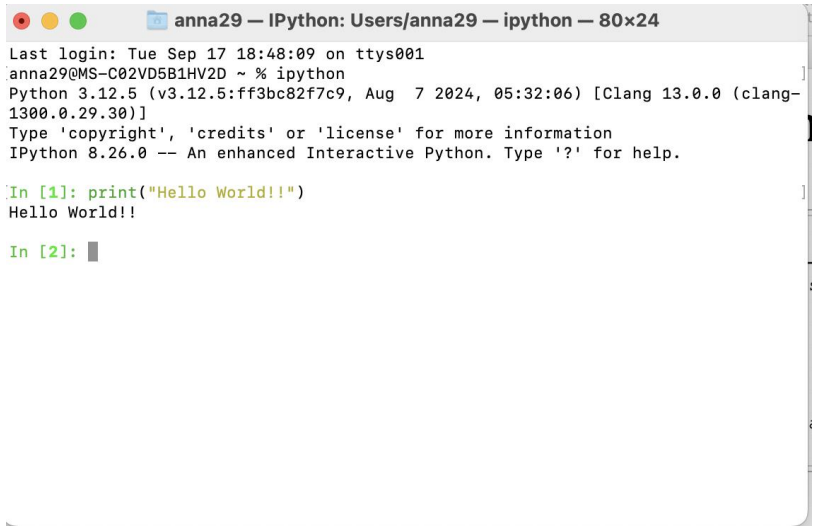
Memory

Memory is to store data and program instructions for CPU to execute. A memory unit is an ordered sequence of bytes.



Programs

Computer *programs*, known as *software*, are instructions to the computer.



Programming Languages

- **Machine language** is a set of primitive instructions built into every computer. The instructions are in the form of binary code. Example: `1101101010011010`
- **Assembly languages** were converting assembly language programs into machine code. Example: ADDF3 R1, R2, R3
- **The high-level languages** are English-like and easy to learn and program. Example: area = 5 * 5 * 3.1415

Guido van Rossum

A Dutch programmer who created Python 1990

d that Python's predecessor, ABC, was inspired by SETL, noting that ABC
ert Meerten YU before comi
C design".[3

, Van Ross wn from the posit
tor for life c

Progra al [edit]

ssum subm nced Research F
) called "Computer Programming for Everybody", in which he further defi

SETL is a very high-level programming language based on the mathematical theory of sets. It was originally developed at the New York University (NYU) Courant Institute of Mathematical Sciences in the late 1960s, by a group containing (Jack) Jacob T. Schwartz, R.B.K. Dewar, and E. Schonberg. Schwa

Elementary programming

Objectives:

- **Print**
- **Input**
- **Variable**

First program

Example:

Text following the hash(#) is a comment

```
# Display welcome message
print("Welcome to CST1510")
```

Function *print*

The built-in **print function** displays its argument(s) as a line of text

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Outputting text using *print* function:

In Python, you have several ways to define and print strings, and the choice between:

- single quotes (**'**),
- double quotes (**"**),
- or triple quotes (**"""** or **'''**, but the *Style Guide for Python Code* recommends **"""**.)
  - multiline strings
  - strings containing single or double quotes
  - **docstrings**—the recommended way to document the purposes of certain program components.

```
#'Between single quotes'
print ('statment: "To know and not yet to do is not yet to know"')

#"Between double quotes"
print ("I'm very happy today")

#"""Using triple-quotes for  strings with multiple  lines"""
print ('''..........
    John Doe
    London
    NW4
...........''')
```

```
statment: "To know and not yet to do is not yet to know"
I'm very happy today
..........
    John Doe
    London
    NW4
...........
```

Outputting text using *print* function:

It may enclose a string in single quotes (’).

```
print('Welcome to Python!')
```

```
Welcome to Python!
```

Pass as a Comma separated (,) Printing a Comma-Separated List of Items

```
print('Welcome', 'to', 'MDX!')
```

```
Welcome to MDX!
```

Concatenate (+) join two strings/texts using the + sign

```
print('Welcome'+ 'to'+ 'MDX!')
```

```
WelcometoMDX!
```

The *print* *function* may contains:

- ***sep*** parameter **,** soft space, feature, which can be made to any character, integer or string as per our choice.

```
#For formatting a date
print('25','09','2024', sep='/')

#Another example
print('a.wieclaw','mdx.ac.uk', sep='@')
```

```
25/09/2024
a.wieclaw@mdx.ac.uk
```

- ***end*** parameter**,** how to print without a newline

```
# Ends the output with a space
print("Welcome to", end=' ')
print("MDX")
```

```
Welcome to MDX
```

Multiple Lines of Text with a Single Statement

- A backslash (\ ) in a string acts as an **escape character**.
- The backslash followed by another character forms an **escape sequence**.
- \n is the **newline character** escape sequence, instructing print to move the output cursor to the beginning of the next line

```
print('Welcome\nto\n\nPython!')
```

```
Welcome
to

Python!
```

Other Escape Sequences in *print*

Escape sequence	Description
`\n`	Insert a newline character in a string. When the string is displayed, for each newline, move the screen cursor to the beginning of the next line.
`\t`	Insert a horizontal tab. When the string is displayed, for each tab, move the screen cursor to the next tab stop.
`\\`	Insert a backslash character in a string.
`\"`	Insert a double quote character in a string.
`\'`	Insert a single quote character in a string.

```
print('this is a longer string, so we \
split it over two lines')
```

```
this is a longer string, so we split it over two lines
```

Comments

Comments are text in a program that are ignored during execution and are used to explain the code. There are two types of comments:

- **Single-line comments** (# Your comment)—useful for adding brief notes or explanations directly in the code.
- **Multi-line comments** (''' Your comment ''')—typically used to provide detailed descriptions of how functions, classes, or other code segments work.

Triple quotes are often use for documentation and or comments

PEP 8 – Style Guide for Python Code

For more information, refer to the https://peps.python.org/pep-0008/

Example of comments

Single-line comments

Multi-line comments

```python
# This is a single-line comment explaining the next line of code
x = 10  # Assign 10 to variable x

# Calculate the sum of two numbers and print the result
y = 5
sum_result = x + y  # Adding x and y

print(sum_result)  # Output the sum of x and y

'''
This function takes two arguments, num1 and num2,
and returns their product. It can be useful for
multiplying any two given numbers in the program.
'''
def multiply(num1, num2):
    return num1 * num2

# Using the multiply function and printing the result
result = multiply(3, 4)
print(result)  # Expected output is 12
```

Text & numbers in *print*

```
#Example of using numbers to print a string multiple times
print("*"* 10)

print("age: ", 40)
print("age: ", (40+5))
print("*"* 10)
```

```
**********
age:  40
age:  45
**********
```

```
print('Sum is', 7 + 3)
```

```
Sum is 10
```

Numbers in *print*

Evaluate the numerical elements

```
print( (20 + 4) / (1 + 3 + 2) )
```

```
4.0
```

Activity

Create a code which can output the following:

```
# Write a print statment with your name and surname
```

```
# Write one print statement containing two strings of arguments:
#'IPhone' and another 'Samsung' and join them with a comma
```

```
# Concatynate the following strings in one print statement:
# "I'm " "18" "years old."
```

```
# Inside the print statment pass the value 5
```

```
# Indide the print statment add the 5 + 6 values
```

```
# Inside the print statment calculate ((19 + 2)/ 3 ) - (5 * 1.5)
```

```
# Add two numbers as a text
```

Important to remember

- Python is cAS senSiTive:

```
1  # Display two messages
2  print("Welcome to CST1510")
3  Print('''Enjoy programming in Python !
4  I hope you will find it fun''')
5
```

‘Print’ ant ‘print’ are not the same

- Indentation matter in Python

```
1  # Display two messages
2  print("Welcome to CST1510")
3      print('''Enjoy programming in Python !
4  I hope you will find it fun''')
5
```

In this case line2 will not work

More about it later!

Getting *input* from the User

Reading input from the console enables the program to accept input from the user.

```
variable = input("Enter a value: ")   # Reads input as a string
```

Store the input

Prompt for the user

assign input to variable

Getting *input* from the User

- You will always need to save the user's input into a variable before using it later.
- The input function will ALWAYS RETURN A STRING/TEXT!!

```
name = input("What's your name? ")
```

```
name
```

```
print(name)
```

Activity

- What may be the problem with this code?

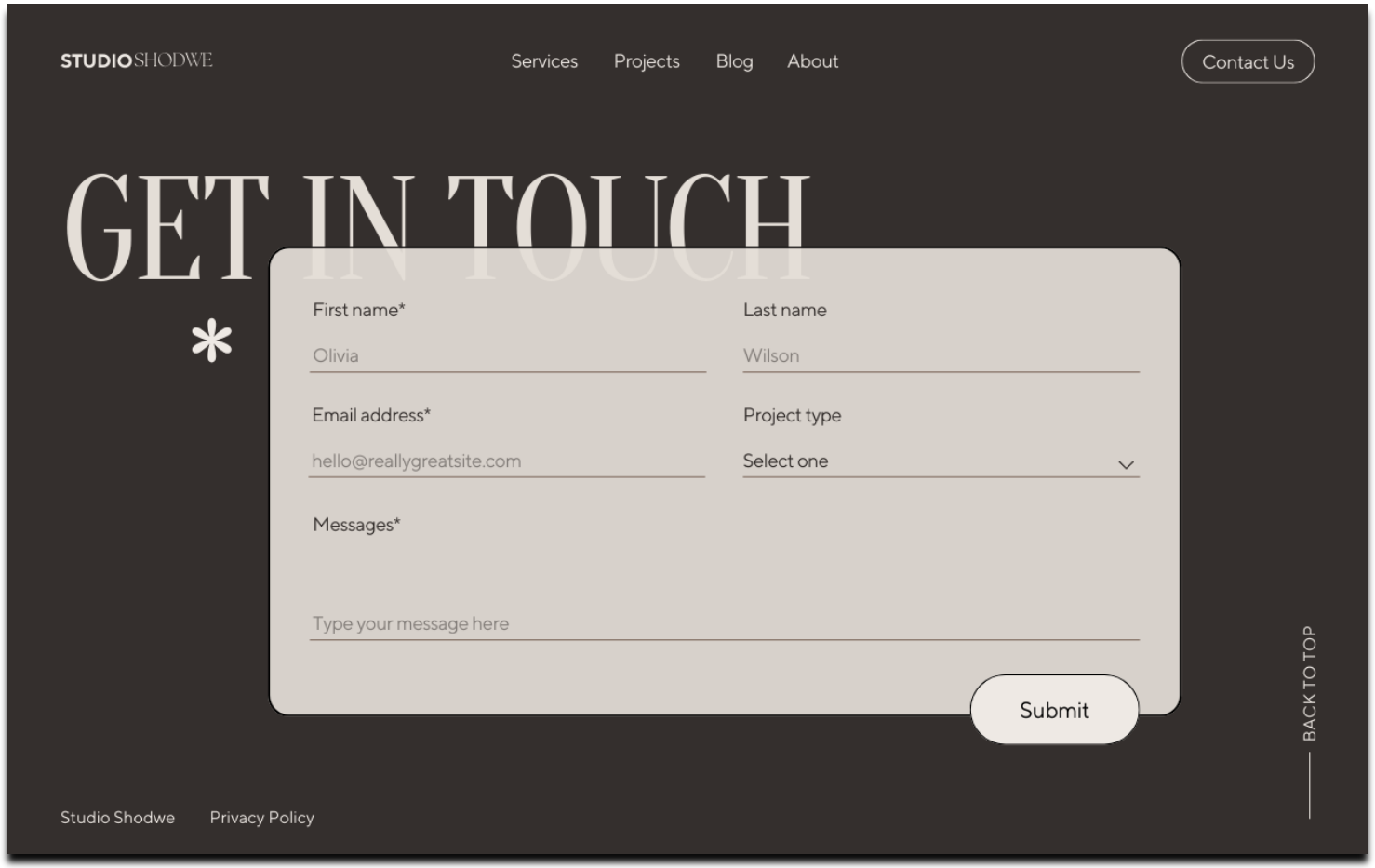
```
value1 = input('Enter first number: ')
```

```
value2 = input('Enter second number: ')
```

```
value1 + value2
```

Activity:

Create a text base version of the “Get In Touch” form

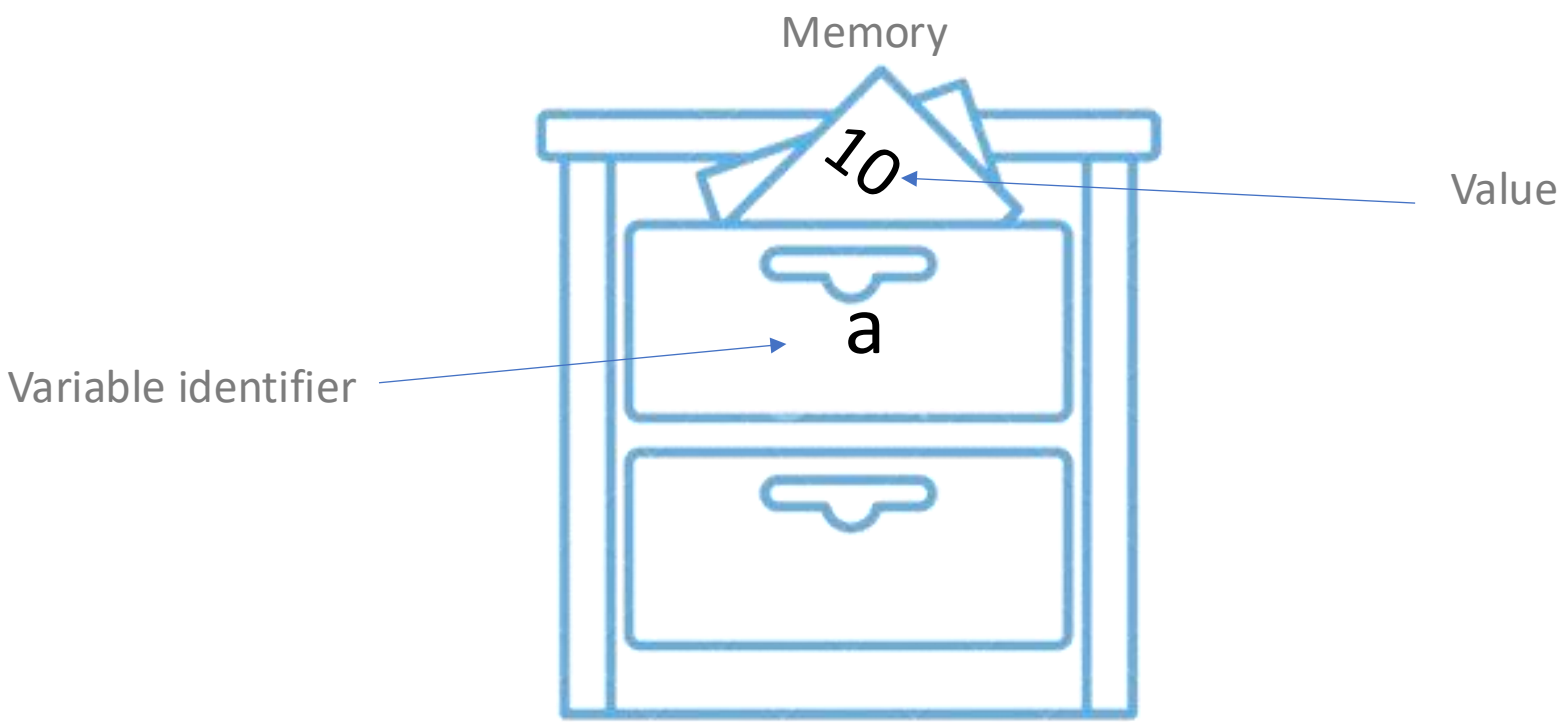


Variable

- A **variable** is a reserved memory location that can be used to store values.
- This means that when you create a variable you reserve some space in the memory

  - Store and retrieve data
  - Use the variable name instead of the value
  - Declare it once and use all the time
  - To possibly change the value if needed
  - Readability, usability

Variable in memory

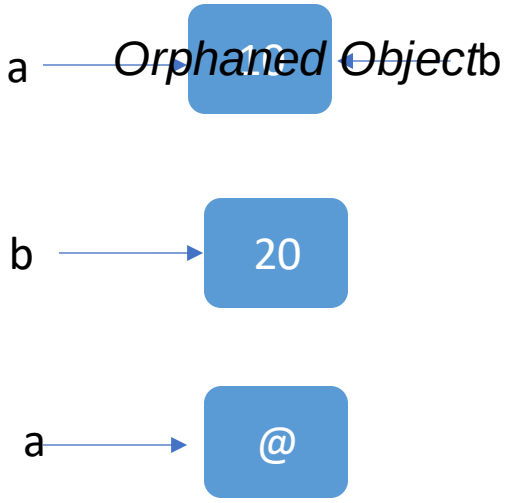


Object references

In Python, all data—including numbers and strings—are objects.

- Variable are reference
- Orphaned Object

```
a = 10
b = a
b = 20
a = '@'
```



Activity

What will be the value of ’x’ and ‘y’ after each line ?

```
x, y = 10, 20     #1 x = ? , y = ?
x = 5             #2 x = ? , y = ?
y, x = x, y       #3 x = ? , y = ?
y = 3             #4 x = ? , y = ?
x = y = x         #5 x = ? , y = ?
```

Variable naming guide:

- Can contain:
  - Lowercase letters (a-z)
  - Upper case letters (A-Z)
  - Digits (0-9) – but cannot start with a digit
  - Underscore (_) – do not use at the beginning as it has special meaning in python

  - Variable names should be descriptive
  - All uppercase variables names for constants(if the value should not change)
  - Short names for temporary variables
  - Longer, meaningful names for others
  - CanUseCamelCaseForSeveralWords
  - can_use_underscores_to_separate_words

Example

```
name = "Anna"

age = 25

PI = 3.14

radius = 10

area_of_circle = PI * radius**2

or = 20
```

Reserve words

You should NOT use them for naming your variables

and	as	assert	break
class	continue	def	del
elif	else	except	exec
finally	for	from	global
if	import	in	is
lambda	not	or	pass
print	raise	return	try
while	with	yield	

Activity

Which of the following identifiers (**variable names)** are valid?

1. Miles =100
2. Test = ‘test1’
3. a+b
4. 4#R
5. #44
6. Apps
7. x_
8. _radius
9. num_value
10. b–a
110. $4
12. elif
13. if
14. y

Variable properties

Variable Name: `a`

- A variable name is an identifier that holds data. In this case, `a` is the name of the variable.

```
a = 500
```

Value: `500`

- The value assigned to the variable a is 500. This is the data that the variable stores.

```
print(a)
```

```
500
```

Memory Address:

- Every variable has a unique memory address where its value is stored. This address can be checked using the `id()` function:

```
print(id(a))
```

```
4623623216
```

Data Type: `int` (Integer)

- The data type of the value 500 is an integer (int), which means it represents a whole number.

```
print(type(a))
```

```
<class 'int'>
```

Variable assignment

Standard

```
num1 = 6                                # assign 6 to num1
```

Simultaneous

```
num1, num2, num3 = 10, 20, 30   # assign 10 to num1, 20 to num2, 30 to num3
```

Chain

```
num1= num2 = num3 = 100         # assign 100  to num1,  num2 and num3
```

Activity

Write a Python program to compute the average age of people in your group.

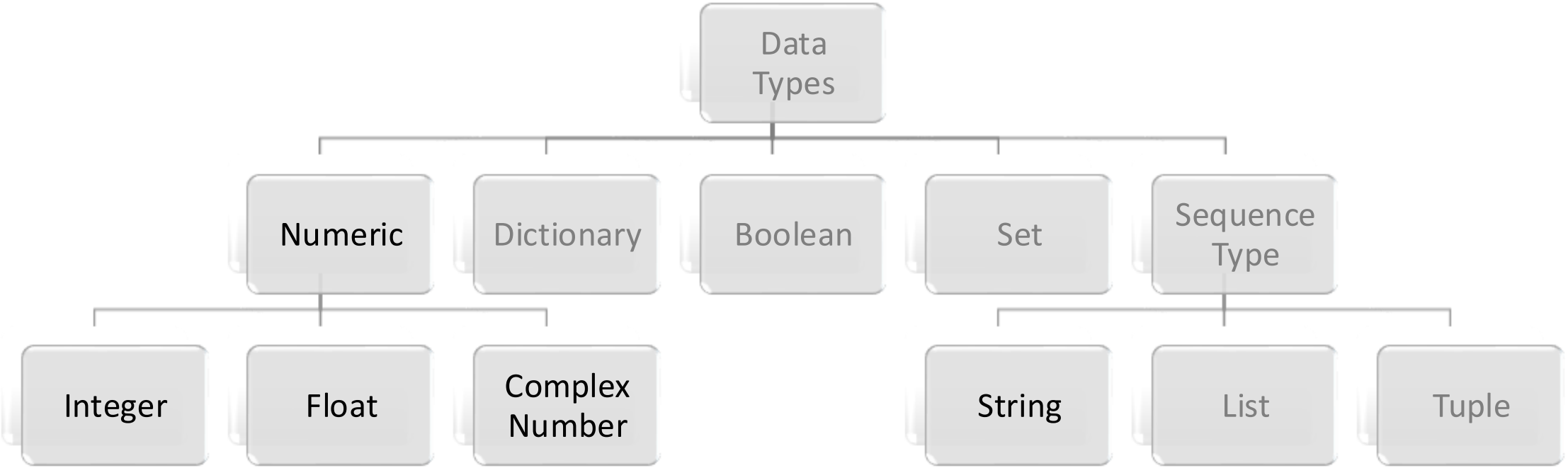
1.Average Calculation:

1. The average is calculated by adding all the ages together and then dividing the sum by the number of people.

2.Steps:

1. Declare a variable for each person's age.
2. Use **simultaneous assignment** to assign ages to each person in one line of code.
3. For each person, **check the data type** to ensure the values are numbers
4. **Print the result** showing the average age.

Python - data types



Integer – whole number

- In Python, integer data consists of a number that does not include a decimal point.

- Examples of integer data:
  - -100
  - -12
  - -1
  - 0
  - 1
  - 20
  - 500

- Integers are <u>not placed in quotation marks</u>. You can use the print() function

Integer – Examples

```
var_1 = -100        # Negative integer
var_2 = -12         # Another negative integer
var_3 = 0           # Zero
var_4 = 500         # Positive integer
var_5 = 1_000_000   # Large integer, using underscores for readability
var_6 = int(1e6)    # 1e6 means 1 * 10^6, but converted to an integer
```

Float - fraction

Float data is data consisting of a number with a decimal point.

- Example:
  - -2.2
  - -1.34
  - 0.0
  - 0.34
- Just like integer data, float data is not placed in quotation marks. In addition, you can also use the print() function to display float data or to perform mathematical calculations with float data. You can run the following code to review the result of this calculation:

```
print(1.2 + 2.8)
```

```
4.0
```

Float - Examples

```
var_1_ = -100.0  # Negative float
var_2_ = -12.5   # Negative float
var_3_ = 0.0     # Zero as a float
var_4_ = 500.75  # Positive float
var_5_ = 1_000_000.0  # Large float using underscores for readability
var_6 = 1e6     # 1e6 means 1 * 10^6, represented as a float
var_7 = 5.5e-3  # 5.5 * 10^-3, small float using scientific notation
```

Complex numbers

- Expressed in the form ax + bj
- Consist of ‘real’ and ‘imaginary’ part
- J is the square root of -1

Assignment:

```
a = 2+3j
b = 3+5j
```

String - text

A string is a sequence of zero or more characters. In Python, string data consists of an ordered sequence of characters. Characters in a string may include letters, numbers, symbols, and spaces. These characters must be placed within quotation marks.

Examples:

- "updates needed"
- "20%"
- "5.0"
- "35"
- "//**"
- ""
- "a"

- Note: The last item (""), which doesn't contain anything within the quotation marks, is called an empty string.

String - Example

```
var_str_1 = "updates needed"  # A simple string containing text
var_str__2 = "20%"             # A string representing a percentage
var_str__3 = "5.0"             # A string that looks like a float
var_str__4 = "35"              # A string that looks like an integer
var_str_5 = "//**"            # A string with special characters
var_str_6 = ""                # An empty string
var_str_7 = "a"               # A single character string
```

Numeric Operators

Operator	Meaning	Example	Result
`+`	Addition	`34 + 1`	`35`
`-`	Subtraction	`34.0 - 0.1`	`33.9`
`*`	Multiplication	`300 * 30`	`9000`
`/`	Float Division	`1 / 2`	`0.5`
`//`	Integer Division	`1 // 2`	`0`
`**`	Exponentiation	`4 ** 0.5`	`2.0`
`%`	Remainder	`20 % 3`	`2`

Example

```
# Integer division - it will describe the fraction
print(5 // 2)
```

```
2
```

```
# Exponentiation. The 1st value is the base, and the second is the power
print( 2 ** 3 )
```

```
8
```

```
# Remainder. Keeps the leftover of the division (Note: we can only put 2 + 2 + 1 into 5, so the remainder is 1)
print(5 % 2)
```

```
1
```

```
# remainder is very useful when we want to know, for example how many seconds are left in a minute
print(100 % 60 ) # 100 sec remainder 60 will tell us that we have 40 sec left
```

```
40
```

```
# remainder is very usefule if we want to know if number is even or odd
# as every even number is devidable by 2 and done leave any left over
print(7 % 2)
```

Activity

Evaluate the following expressions:

```
4 / 2
```

```
2 / 4
```

```
5 // 2
```

```
2 // 4
```

```
2 ** 3
```

```
(10 + 6) % 7
```

Augmented Assignment operators

Operator	Name	Example	Equivalent
`+=`	Addition assignment	`i += 8`	`i = i + 8`
`-=`	Subtraction assignment	`i -= 8`	`i = i - 8`
`*=`	Multiplication assignment	`i *= 8`	`i = i * 8`
`/=`	Float division assignment	`i /= 8`	`i = i / 8`
`//=`	Integer division assignment	`i //= 8`	`i = i // 8`
`%=`	Remainder assignment	`i %= 8`	`i = i % 8`
`**=`	Exponent assignment	`i **= 8`	`i = i ** 8`

Examples

```
a = 10
print('a = ',a)
```

```
a =  10
```

```
a += 5
print('a += 5 is ',a)
```

```
a += 5 is  15
```

```
a -= 5
print('a -= 5 is ',a)
```

```
a -= 5 is  10
```

```
a *= 5
print('a *= 5 is ',a)
```

```
a *= 5 is  50
```

```
a /= 2
print('a /= 2 is ',a)
```

```
a /= 2 is  25.0
```

```
a //= 2
print('a //= 2 is ',a)
```

```
a //= 2 is  12.0
```

```
a %= 2
print('a %= 2 is ',a)
```

```
a %= 2 is  0.0
```

```
a = 5 * a + 6
print('a = 5 * a + 6 is ',a)
```

```
a = 5 * a + 6 is  6.0
```

Activity

Assume that `a = 5`, and that each expression is independent.
What are the results of the following expressions?

```
a += 5
a -= 5
a *= 5
a /= 2
a //= 2
a %= 2
a = 5 * a + 6
```

Casting

Casting is a conversion of one data type into another. In Python is done using constructor functions:

- **int()** - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)

```
# Casting from string to int
user_input = '10'
print(int(user_input))
```

```
10
```

- **float()** - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)

```
# Casting from float to int

print(int(13.5))
```

```
13
```

- **str()** - constructs a string from a wide variety of data types, including strings, integer literals and float literals

```
# Casting from int to string
str(15)
```

```
'15'
```

Activity

Write a Python program that asks the user to input two numbers. The program should:

1. **Prompt the user** to enter two numbers (as strings).
2. **Cast** the user inputs to integers or floats.
3. Perform the following operations on the two numbers:
    1. Addition
    2. Subtraction
    3. Multiplication
    4. Division
4. Print the result of each operation.

Build in functions

Function	Description	Example
`abs(x)`	Returns the absolute value of `x`.	`abs(-2)` is `2`
`max(x1, x2, ...)`	Returns the largest among `x1, x2, ...`.	`max(1, 5, 2)` is `5`
`min(x1, x2, ...)`	Returns the smallest among `x1, x2, ...`.	`min(1, 5, 2)` is `1`
`pow(a, b)`	Returns `a` raised to the power of `b`. Same as `a ** b`.	`pow(2, 3)` is `8`

`round(x)`	Returns the integer nearest to `x`. If `x` is equally close to two integers, the even one is returned.	`round(5.4)` is `5`
		`round(5.5)` is `6`
		`round(4.5)` is `4`

`round(x, n)`	Returns the float value rounded to `n` digits after the decimal point.	`round(5.466, 2)` is `5.47` `round(5.463, 2)` is `5.46`

Examples:

```python
# 1. Absolute Value of a Number
print("Absolute value of -3:", abs(-3))
```

```
Absolute value of -3: 3
```

```python
# 2. Maximum Value
print("Maximum value in the list:", max(2, 5, 3, 8, 10))
```

```
Maximum value in the list: 10
```

```python
# 3. Minimum Value
print("Minimum value in the list:", min(2, 5, 3, 8, 10))
```

```
Minimum value in the list: 2
```

```python
# 4. Power Calculation (e.g., 2 raised to the power of 3)
print("2 raised to the power of 3:", pow(2, 3))
```

```
2 raised to the power of 3: 8
```

```python
# 5. Rounding Numbers
print("Rounding 5.4:", round(5.4))
print("Rounding 5.5:", round(5.5))
print("Rounding 4.5:", round(4.5))
```

```
Rounding 5.4: 5
Rounding 5.5: 6
Rounding 4.5: 4
```

```python
# 6. Rounding with Decimal Places
print("Rounding 5.466 to 2 decimal places:", round(5.466, 2))
print("Rounding 5.463 to 2 decimal places:", round(5.463, 2))
```

```
Rounding 5.466 to 2 decimal places: 5.47
Rounding 5.463 to 2 decimal places: 5.46
```

Formatting strings & floats

String formatting:

```
name = 'Sean Connery'
```

```
print (f'My name is {name}')  # f-Strings (Formatted String Literals)
```

```
My name is Sean Connery
```

```
#Examle of string formatting using C style and the format specification

print ('Hello %s' % name ) # C-Style String Formatting (%)
print ('My name is {1}, {0} {1}'.format('James', 'Bond'))  # str.format() Method
```

Float formatting:

```
pi = 3.14159
print('Pi to 2 decimal places: %.2f' % pi) #%.2f: Limits the float to 2 decimal places in C-style formatting
```

```
Pi to 2 decimal places: 3.14
```

```
print('Pi to 2 decimal places: {:.2f}'.format(pi)) #{:.2f}: Similarly limits the float to 2 decimal places using both
```

```
Pi to 2 decimal places: 3.14
```

```
print(f'Pi to 2 decimal places: {pi:.2f}') # .format() and f-strings.
```

```
Pi to 2 decimal places: 3.14
```

Activity

Create a simple program that will help you automate the task of computing for your **restaurant bill**. Assuming your total bill covers the **basic meal cost**, **sales tax**, and **tip**, you will need the following information to start working on your program.

- basic meal cost ?
- sales tax rate 7% of basic meal cost
- tip 20% of the sum of the basic meal cost and sales tax

- Print out the bill using the format string & floats

Modules vs. Libraries in Python

- **Module:** A single file of Python code.
- **Library:** A collection of related modules offering extended functionality.

Some libraries, like the built-in **math** library, are ready to use without installation. However, others, like **pandas**, may need to be installed first. For guidance on installing libraries, refer to the **Python installation and setup** instructions.

Example

```
#Example
import math
math.sqrt(120)
```

```
10.954451150103322
```

```
# or by explicitly importing the required function
from math import sqrt
sqrt(120)
```

```
10.954451150103322
```

Mathematical functions

Function	Description	Example
`fabs(x)`	Returns the absolute value of `x` as a float.	`fabs(-2)` is `2.0`
`ceil(x)`	Rounds `x` up to its nearest integer and returns that integer.	`ceil(2.1)` is `3` `ceil(-2.1)` is `-2`
`floor(x)`	Rounds `x` down to its nearest integer and returns that integer.	`floor(2.1)` is `2` `floor(-2.1)` is `-3`
`exp(x)`	Returns the exponential function of `x` (e^x).	`exp(1)` is `2.71828`
`log(x)`	Returns the natural logarithm of `x`.	`log(2.71828)` is `1.0`
`log(x, base)`	Returns the logarithm of `x` for the specified base.	`log(100, 10)` is `2.0`
`sqrt(x)`	Returns the square root of `x`.	`sqrt(4.0)` is `2`
`sin(x)`	Returns the sine of `x`. `x` represents an angle in radians.	`sin(3.14159 / 2)` is `1` `sin(3.14159)` is `0`
`asin(x)`	Returns the angle in radians for the inverse of sine.	`asin(1.0)` is `1.57` `asin(0.5)` is `0.523599`
`cos(x)`	Returns the cosine of `x`. `x` represents an angle in radians.	`cos(3.14159 / 2)` is `0` `cos(3.14159)` is `-1`
`acos(x)`	Returns the angle in radians for the inverse of cosine.	`acos(1.0)` is `0` `acos(0.5)` is `1.0472`
`tan(x)`	Returns the tangent of `x`. `x` represents an angle in radians.	`tan(3.14159 / 4)` is `1` `tan(0.0)` is `0`
`degrees(x)`	Converts angle `x` from radians to degrees.	`degrees(1.57)` is `90`
`radians(x)`	Converts angle `x` from degrees to radians.	`radians(90)` is `1.57`

Activity

- Write a program calculating a distance between this two points. Asking user for the coordinates, perform the calculation and print out the result (using formatting).

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Activity

Write a program calculating how many hours will you spend on python studies during the 2024-2025 academic year.

Few information are given:

- We have 12 weeks of study
- You will have 3h Lab 2h lab a week
- You will practice 2h every day

What are the steps you will have to take to approach the problem

Summery

- Print
- Input
- Variables